

production-readiness

From prototype to the actual implementation

[ARCHITECTURE.md](#) ranks the *architectural* weaknesses of the code as it stands. This file is the other question: **what has to be true before this runs the real migration**, with real contracts, real data and real money at stake.

Ordered by what blocks what. Effort is XS/S/M/L; "blocker" means the real migration cannot run without it.

Update 2026-08-22. [docs/PLAN-CHANGES-22082026.md](#) wired the Target System confirmation stream. A new scale item (2.7) covers the confirmation-topic read being a full scan per run — fine at prototype volume, needs a bounded/incremental read before real volume. Cross-ref §2.1 (FileLoadsBigQueryWriter) and §2.4 (quota review).

Update 2026-08-21. [docs/PLAN-CHANGES-21082026.md](#) has landed: the engine has two dispositions (WRITTEN / REJECTED), one full snapshot per run (no initial/delta, no window), and homogeneous TDS definitions. That changes the answer to several items below: the dedup scale item (2.3) and the excluded-records business-sign-off item (§8) are gone; the replay-idempotency item (§7) rests on count-then-DELETE per `run_id`, not a dedup key; the cutover has no delta chain. Items left as historical (struck-through ) record what was true when written and are not rewritten.

0. What is deliberately a mock today

The prototype's honesty is its strength — but the mocks are the first inventory to work through, because each is a real integration waiting to happen.

Today	Becomes	Owner	Risk if left late
harness/ synthetic generator	Real mainframe extract	Extraction team	Layouts and volumes differ from every assumption below
apps/extractor-app (mock)	Real extractor on the mainframe lane	Extraction team	Artefact naming/ . FLG timing is agreed by convention only (weakness #1)
apps/loader-app (mock)	Real loader against Target System APIs	Loader team	Rate limits, auth and error taxonomy are unknown
apps/target-system-mock	Target System	Vendor	Idempotency semantics assumed, not confirmed. Now deployed on Cloud Run so the GCP Load lane has a target — swapping in the real endpoint is a TARGET_SYSTEM_URL change
Throwaway PGP keypair (make keys)	Managed keypair + rotation	Platform	Key rotation mid-migration is a live-run incident
No CI (.github/ does not exist)	Pipeline that runs make verify per commit	Us	Every guarantee in the deck is currently proved by hand

First action, cheap and high value: turn each mock into a *contract test* against the real counterpart as soon as the other team can expose one — even a staging endpoint. The mocks stay; they become the fast local double, and the contract test proves the double still matches.

1. Correctness and audit — blockers

These are the items where "the prototype proves it" and "an auditor accepts it" differ.

#	Item	Why it blocks	Effort
1.1 ✓	Per record lineage table — done: <code>bq_recon.record_lineage</code> , written by both stages that settle a disposition, guarded by acceptance criterion 7	Today the answer to "why was this account not migrated?" is a <i>count</i> . For a regulated core-banking migration that is not an answer. The <code>EXCLUDED / DUPLICATES</code> tagged outputs already exist at <code>pipeline.py:248,284</code> and are dropped — they only need a sink. (weakness #5)	S
1.2 ✓	Recon reads ledgers, not COUNT(*) — done: one ledger read plus a ledger-vs-register agreement check that fails the run. The remaining counts cover what the ledger does not	Recon re-derives in hand-written SQL what <code>run_ledger</code> already recorded, so the two can disagree and nothing says which is right. Recon's real job is the part aggregates <i>cannot</i> do: key-level orphans (target rows with no source) and cross-checking the extractor/loader <code>.RPT</code> claims. (weakness #4)	M
1.6	Name evidence columns for the stage that produced them	<code>run_ledger.target_written</code> counted rows into <code>account_src</code> while being read as "reached Target System". Renamed to <code>extraction_written</code> before any dashboard depended on it — the window for a free rename closes the moment an external consumer appears	XS
1.3 ✓	Shared contract manifest across Python and Java — done: <code>contracts/artefacts.json</code> carries artefact naming and the shared BQ columns, loaded by both languages. The <code>Stage / Reason</code> taxonomy is still Python-only	A rename in Python is invisible to Java until a run fails at integration — and at real volume that failure costs a run window, not a test. (weakness #1)	S
1.4	Balance failure must page a human	<code>make verify</code> catches an imbalance on a laptop. In production nobody is watching a terminal: log-based metric on <code>balanced=false</code> + alert, with the run halted, not continued.	S
1.5	Signed, immutable evidence	Reports and ledgers are the audit trail. Write them to a bucket with retention/lock, hash them, and record the hash in <code>run_ledger</code> , so evidence cannot be silently re-generated.	M

2. Scale — the prototype has never seen the real numbers

Target is ~1.7M accounts / ~20B transactions. The prototype runs 405–505 records.

#	Item	Why	Effort
2.1 ✓	Replace insertAll with load jobs — done: FileLoadsBigQueryWriter stages NDJSON in GCS and loads it; small writes still stream. Needs a volume test (2.5) to size the threshold	sinks.py:35-65 has two writer classes with identical bodies; both stream row-by-row. Streaming 20B rows is neither affordable nor fast, and insertAll quotas will stop the run. This is the single most important code change on the list. (weakness #3)	M
2.2	Partition + cluster every large table	Partition account_src / account_curated by run id, cluster by account_key (was dedup_key). Already present in terraform/modules/bigquery; the pipelines' ensure_table calls must match, or the tables are created unpartitioned by the first run that wins the race.	S
2.3	Size the GroupByKey shuffle — moot: the dedup stage is removed (see PLAN-CHANGES-21082026.md D1).	Dedup at 20B records is the expensive step. Needs a partitioned run window, a measured shuffle volume, and a decision on whether dedup happens per window or globally — with the survivor rule proven stable across windows. With no dedup stage there is no shuffle to size; the cost moves to the write path (2.1) and the per-run-id DELETE that makes replays idempotent.	L
2.4	Quota review before the first large run	Dataflow CPU per region; BigQuery load jobs capped at 1,500/table/day , which is a real ceiling if batches are small; and SSD_TOTAL_GB , default 500 GB, which Composer 2's own five Autopilot nodes consume entirely — on 2026-08-19 that alone made every DAG pod unschedulable, reported as a pod-start timeout. All need increases requested weeks ahead.	S
2.5	Load/soak test at 1–5% of volume	The only way any of the above is more than a guess. Should run the full DAG, not a single pipeline, and produce a cost figure per 1M accounts.	M
2.6	Batch sizing end to end	The 200-element TARGET batch is a Loader contract, not a scale decision. Confirm it survives 1.7M accounts, or negotiate it.	S
2.7	Bound the confirmation-topic read	Recon reads target-system-confirmations with auto.offset.reset=earliest under a fresh recon- <runId> group every run — a full scan end-to-end. At prototype volume (hundreds of rows) that is instant; at 1.7M accounts it reads every confirmation ever published each run. Needs a bounded/incremental read before real volume: seek-by-timestamp to the run's start, or a compacted topic keyed by account_key with recon reading only the compacted tail. Sits alongside §2.1 (the write path) and §2.4 (the quota ceiling).	M

3. Real contracts and real data

#	Item	Why	Effort
3.1	Copybook → TDS generation	<code>contracts/copybooks</code> exists; the real layouts will arrive as COBOL copybooks and hand-transcription at scale is an error source. Generate the TDS defs, and diff-test the generator against the hand-written ones already in the repo.	M
3.2	Contract versioning	A mapping change mid-migration must be attributable: version each contract, record the version in <code>run_ledger</code> , and refuse a re-run whose contract version differs from the version its first run recorded, unless explicitly forced.	S
3.3	Schema evolution rules	What happens when the source adds a field mid-migration? Today: <code>MAP_UNKNOWN_TRANSFORM</code> or silent pass-through. Decide, encode as a reason code, test it.	S
3.4	Reference-data ownership and freshness	<code>contracts/reference</code> is static in the prototype. In production it has an owner, a refresh cadence, and an effective-date — enrichment joined against stale reference data is a silent correctness bug that balances perfectly.	M
3.5	PII handling and masking at ingest	The moment real banking data reaches a non-prod environment it needs stable pseudonymisation — stable, so reconciliation still works across environments.	M

4. Integration contracts to close with the other teams

Each of these is currently an assumption in code. Each should become a written agreement plus a contract test.

- **Kafka batching is ambiguous** — "batches of 200" could mean 200 messages per produce request or one message holding an array of 200. `sinks.py` implements the former and says so in its docstring. **Ask the Loader team; the wrong reading is a silent data-shape mismatch.**
- **Target System:** authentication (OAuth vs static creds), rate limits, bulk endpoints, retry semantics, the exact idempotency guarantee behind `X-Idempotency-Key`, and the error taxonomy to map onto our `Reason` enum.
- **Extractor lane:** artefact naming, `.FLG` -written-last guarantee, `.CHS` algorithm, PGP recipient and key rotation, delivery channel (SFTP vs direct to the landing bucket), and what a partial extract looks like.
- **Loader lane:** what the Loader does with a record it cannot post, and how that comes back to us so the balancing equation still closes across the whole chain rather than just our half.
- **Downstream consumers:** is the enriched-JSON Kafka topic a contract, and who owns its schema?

5. Security and compliance

#	Item	Effort
5.1	CMEK on buckets and datasets , VPC Service Controls perimeter, confirmed data residency	M
5.2	Least-privilege IAM — the bootstrap currently wants <code>roles/owner</code> ; production wants the narrow set already sketched in <code>terraform/modules/iam</code>	M
5.3	Secret rotation — PGP keypair, Target System credentials, Git token; rotation must not require a code deploy	S
5.4	Audit log retention and export — Cloud Logging sinks with a retention matching the audit duty, not the default	S
5.5	Separation of duties — who may trigger a production run, who may approve an imbalanced run, who may read raw source data	S

6. Delivery — there is no CI today

#	Item	Why	Effort
6.1	CI on every commit : <code>make test</code> + <code>make run-initial</code> + <code>make verify</code> + <code>make verify-project2</code> on the local stack	Every claim in the deck is currently proved by hand. The local stack exists precisely so CI can prove them in minutes with no cloud spend.	M
6.2 ✓	Java unit tests — done : 9 tests in <code>apps/common</code>	<code>mvn -B test</code> runs none. A rename in <code>Artefacts.java</code> is caught only at E2E, if at all. (weakness #7)	M
6.3	dev / stage / prod environments with separate state and separate projects	Today <code>terraform/envs/dev</code> is the only env; promotion is not a thing that can be rehearsed. (weakness #6)	M
6.4 ✓	Required Terraform inputs — done differently : a plan-time <code>check</code> on the Dataform credential pair, and the real Dataflow subnet/SA emitted in <code>env_file</code> instead of guessed	So an omission fails at <code>plan</code> , not when a job hangs mid-run. (weakness #6)	S
6.5	Image build in the cloud, pinned by digest	Images are built on a laptop and pushed today; the DAG then runs "whatever <code>:latest</code> is". A production run must be reproducible from a digest recorded in <code>run_ledger</code> .	M
6.6	Link the Dataform repository to git and switch the DAG back to the Dataform operators	The <code>dataform_run</code> pod exists only because the repo is unlinked.	S

6b. What only the DAG path can prove

The lesson from the first real Composer run: **smoke-gcp and the DAG authenticate as different identities**, so a whole class of defect is invisible until the DAG runs. The smoke path uses the operator's own credentials; the DAG's pods use Workload Identity as `dataflow-worker`, which is least-privilege by design. The first DAG run failed on a `storage.buckets.create` that no correctly-scoped service account would ever be granted.

Anything that touches IAM, quotas or the distributed runner therefore needs a DAG run to be considered tested — a green `make smoke-gcp` is necessary and not sufficient. That is an argument for 6.1 (CI) growing a scheduled DAG run against a `stage` environment, not just the local-stack checks.

The 2026-08-23 Kafka run made the same point five more times. Every one of these was green locally, green in unit tests, and broken on GCP: a Composer environment variable that never reached the pods (a `KubernetesPodOperator` passes only what `env_vars` names); a `confluent_kafka_oauth_cb` with the wrong arity, never called; a bare access token where Managed Kafka wants a JWT-shaped envelope; `kafka-clients` 3.7 calling a JDK API that JDK 25 removed; and — the one that cost the most — a *deployed component no build target ever rebuilt*, so four correct fixes shipped everywhere except the process that needed them. Redpanda in the local stack is PLAINTEXT, which is precisely why none of the four authentication defects could appear there. Two durable lessons for §6.1:

- **CI must exercise an authenticated broker**, not just a PLAINTEXT one, or the entire SASL path stays untested until a human runs it by hand.
- **Every deployed artefact needs exactly one build path that CI runs.** An image that only a Dockerfile knows how to build, and that no `make` target names, will silently rot in the registry while the repo moves on — and nothing in the test suite can see it.

A third, cheaper lesson: keep an SLF4J binder on the classpath of anything using `kafka-clients`. Four of those five defects presented as the same content-free `TimeoutException: Timeout expired while fetching topic metadata`, because the client's own explanation was being discarded by the NOP logger.

7. Operability

- **Monitoring:** log-based metrics on reject counts by reason, on imbalance, on per-stage duration; a dashboard the operator watches during a run window.
- **Alerting:** balance failure, DAG task failure, run overrunning its window, Target System error rate above threshold.
- **Runbook for a failed run:** how to resume, what is safe to replay (replays are idempotent by `run_id` — a re-run **replaces** that run's rows via count-then-DELETE and touches no other run; that property must be *tested at volume*, not assumed), and how to abandon a run cleanly.
- **Airflow tuning:** pools and concurrency so three pipelines cannot exhaust Dataflow quota, retries with backoff, SLA misses that alert.
- **Dead-letter path:** today a poison record becomes a `REJECTED` row; confirm that is still the right behaviour when 0.01% of 20B records is 2M rejects.
- **Cost control:** budget alerts, Dataflow autoscaling caps, and a decision on whether Composer stays up for the migration period (~\$300–400/mo) or is created per run window.

8. Cutover

- **Rehearsals** — at least two full dress rehearsals on production-shaped data, with the reconciliation reports reviewed by the people who will sign them off for real.
- **Run windows** — agreed freeze windows with the mainframe team; each run is one full snapshot of the source, so a single run's read → write must fit inside its window, which is what §2.5 measures. (There is

no initial-plus-delta chain — one snapshot per run.)

- **Dual-run reconciliation** — a period where both cores hold the accounts and a daily recon proves they agree; this is what makes the go/no-go decision evidence-based.
- **Abort and rollback criteria, written in advance** — what imbalance, what reject rate, what Target System error rate stops the migration. Decided before the run, not during it.
- **Business sign-off on the migrability report** — every record leaves through one of two doors (WRITTEN or REJECTED); the rejected records are a business decision, not a technical one, and each is named in record_lineage (§1.1). With the contract-exclude filter removed, "should not migrate" is a source-feed concern, not an engine stage — so the list to sign off is the reject log, not a separate excluded set.

Suggested sequence

Phase	Contains	Exit criterion
A — Make correctness auditable	1.1 ✓, 1.2 ✓, 1.3 ✓, 1.4, 6.1, 6.2 ✓, 6.4 ✓	CI proves all 8 criteria per commit; every not-migrated record is named, not just counted
B — Make it survive volume	2.1 ✓, 2.2, 2.4, 2.5, 2.7	A 1–5% soak run completes inside a window, with a cost per 1M accounts
C — Make it real	0 (mocks → contract tests), 3.x, 4.x, 6.3, 6.5, 6.6	The chain runs against the other teams' real endpoints on stage
D — Make it safe to run for real	5.x, 7.x, 1.5, 8.x	Two clean rehearsals; abort criteria signed; dual-run recon agrees

Phase A is worth starting before the real contracts arrive — none of it depends on them, and it is what makes the arrival of real data measurable rather than alarming.